IT4931 / IT5384 / IT4043E · SOICT · HUST

Apache Kafka

Lecture 6 — Distributed messaging

Topics · partitions · producers · consumers · Connect / ksqlDB

Learning outcomes (CLO2, CLO3)

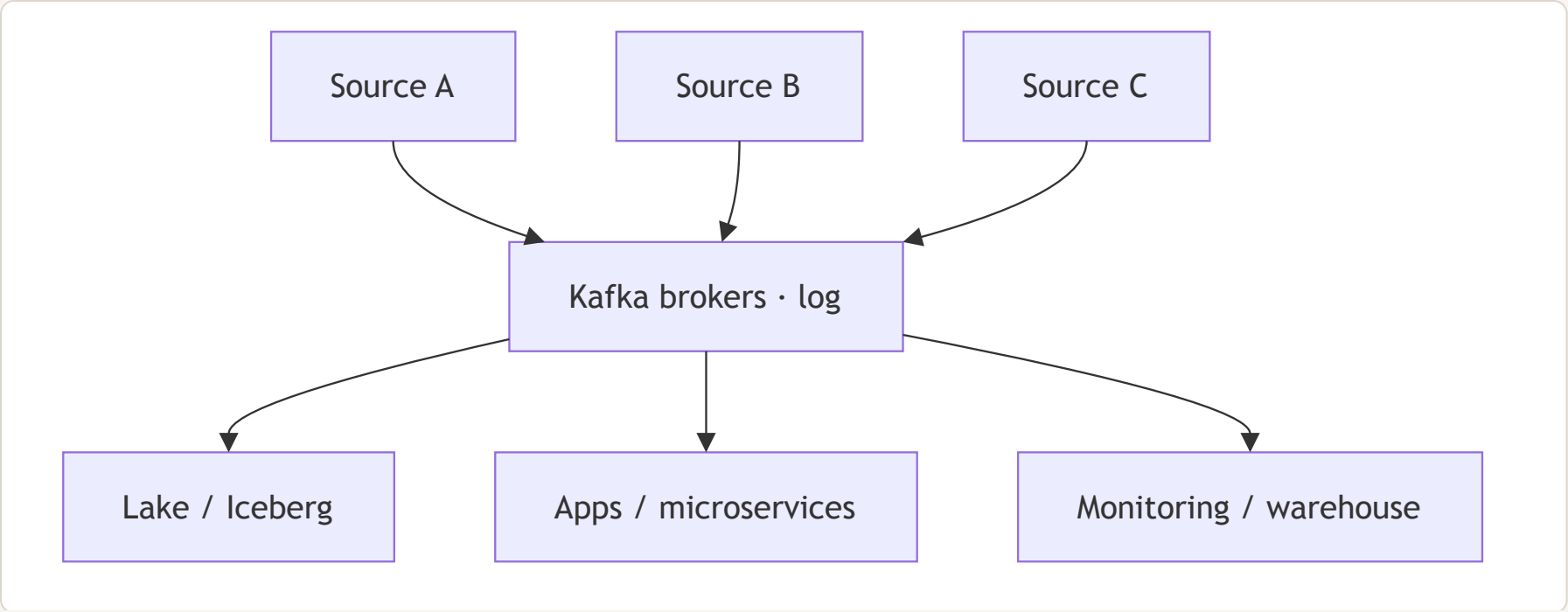
1. **Explain** Kafka as a distributed **event / commit log** that decouples producers & consumers
2. **Describe** topics, partitions, offsets, brokers, replication (leader / ISR)
3. **Reason** about producer choices: partition key, acks, batching
4. **Explain** consumer groups, offsets, rebalance, at-least-once / EOS design
5. **Sketch** an end-to-end topic (e.g. `orders`) and place Connect / ksqlDB in the stack

Roadmap

1. **6.1** Introduction & use cases
2. **6.2** Architecture — topics, partitions, brokers, replication, KRaft
3. **6.3** Publishers (producers)
4. **6.4** Consumers & groups
5. **6.5** Ecosystem — Connect · CDC · ksqlDB

6.1 Introduction

Why Kafka? Decouple pipelines



Producers **don’t know** consumers · clients track **offsets** · higher throughput than classic MQ for event streams.

What is Kafka?

Property	Meaning
Pub/sub streams	Topics as named feeds of records
Durable log	Disk-backed, replicated partitions
Fast I/O	Sequential writes, batching, compression, zero-copy
Scale-out	Partition across brokers

Records: optional **key**, **value**, **timestamp** — **immutable** once written.

What you can build

- Real-time apps reacting to streams (metrics, CEP, fraud)
- Feed analytics / Spark / lakes (high-latency batch still OK)
- Shared **commit log** between services (resync state)
- Live dashboards & summaries

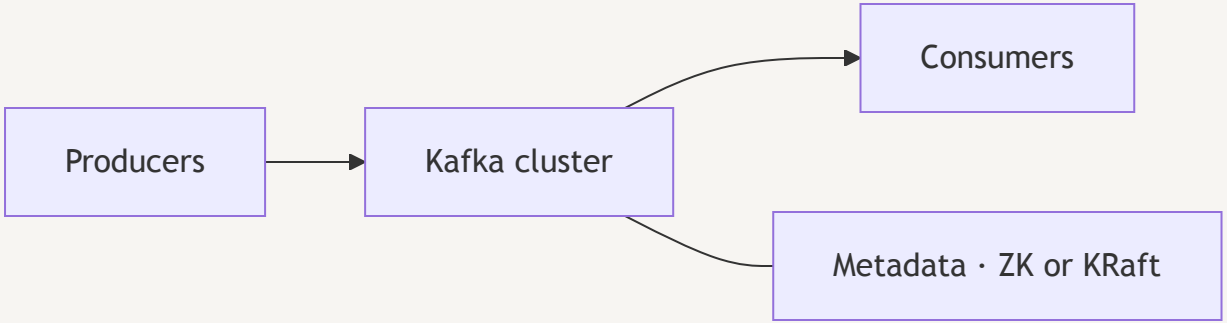
Industry adoption (examples)

Banking fraud · retail inventory · insurance monitoring · IoT queues · Uber matching · Twitter recommendations · Netflix event backbone · Walmart inventory streams

In many cloud data platforms, Kafka is the default event bus.

6.2 Architecture

Kafka overview



Cluster of **brokers** holding topic partitions; clients discover leaders via metadata.

Topics, producers, consumers



Topic = named feed · producers append · consumers pull at their own pace.

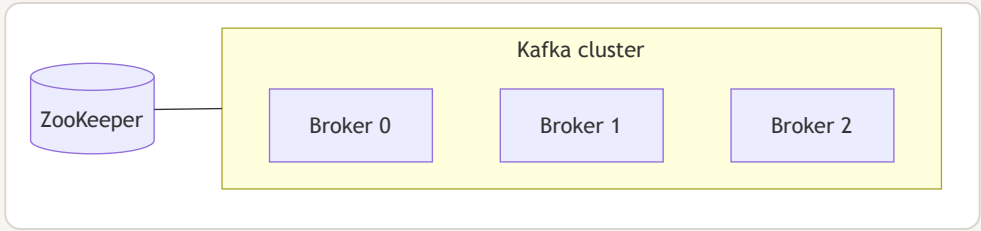
Key terminology

Term	Role
Topic	Named stream / category of records
Partition	Ordered shard of a topic log
Offset	Sequential id within a partition
Producer	Publishes (appends) records
Consumer	Reads the feed
Broker	Server in the Kafka cluster

Architecture — ZooKeeper era

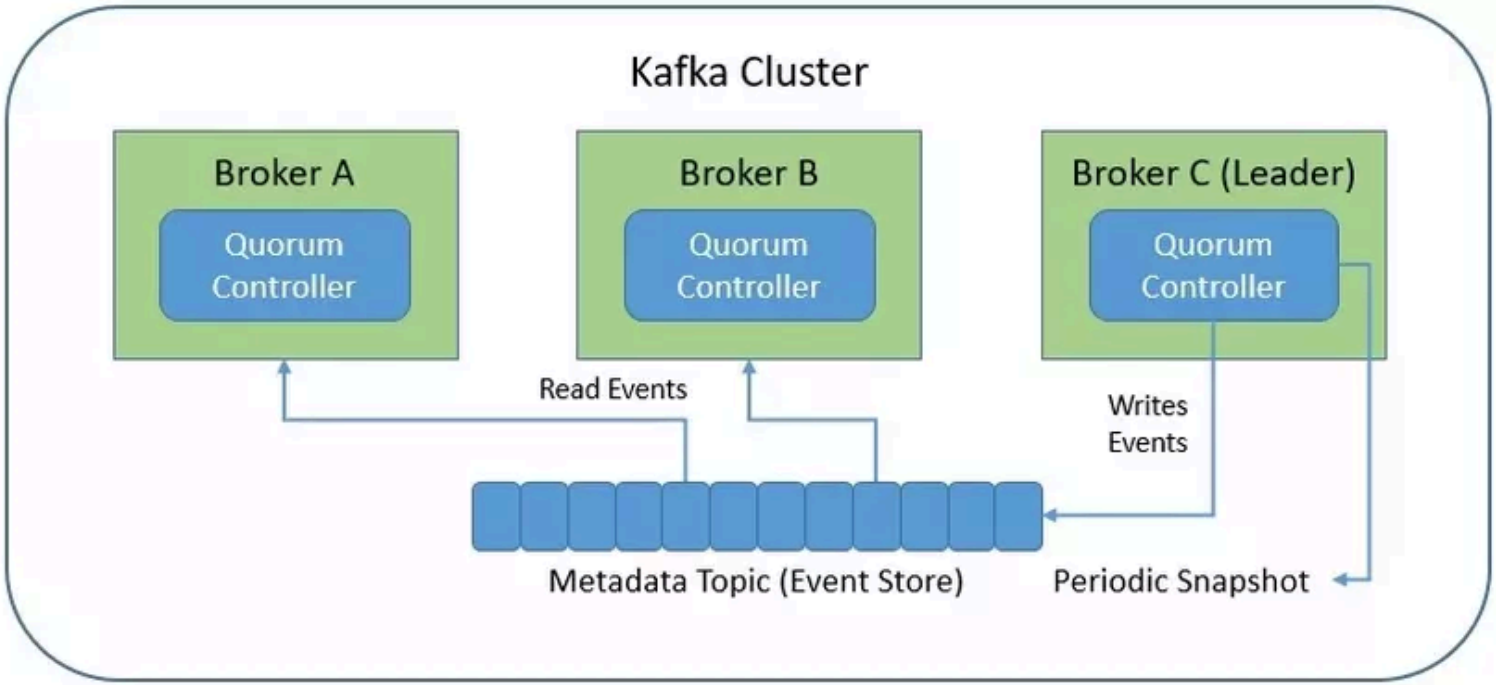
Classic path: brokers + **ZooKeeper** for cluster metadata.

- Membership / service discovery
- Controller election
- Topic config changes

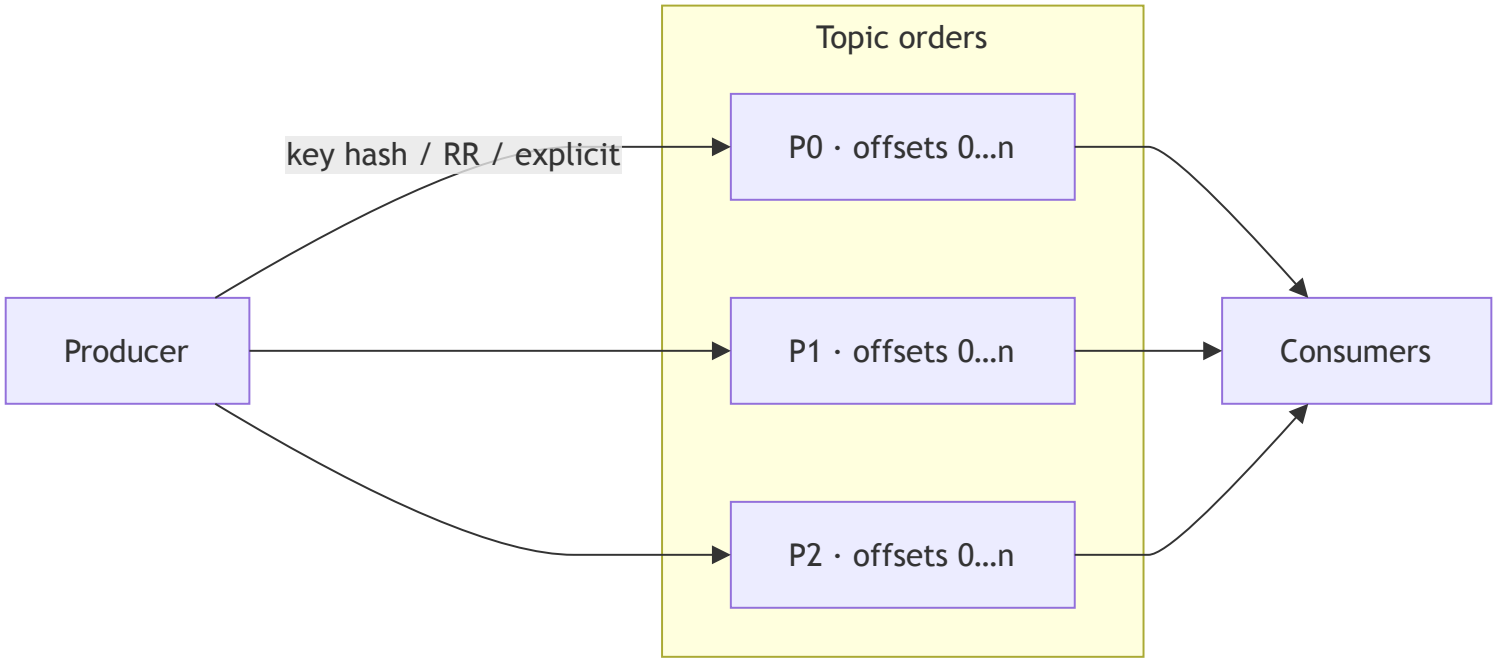


KRaft — ZooKeeper-less metadata

KRaft: Kafka's own consensus for metadata — simpler ops in modern clusters (2026 default path).

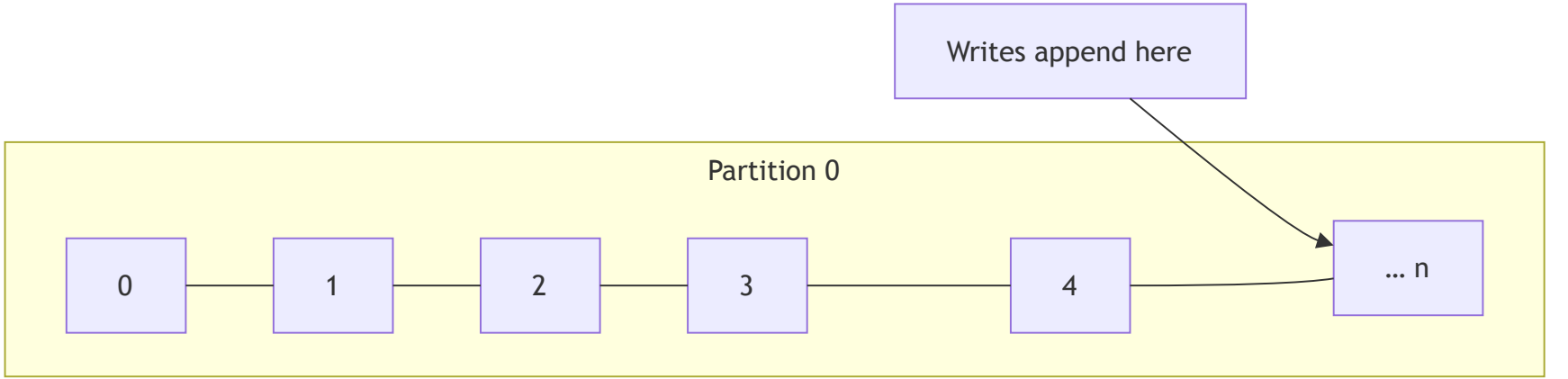


Topic = partitioned commit log



Topic = enclosure of partitions · Producer appends to a **chosen partition** · Order **per partition** only.

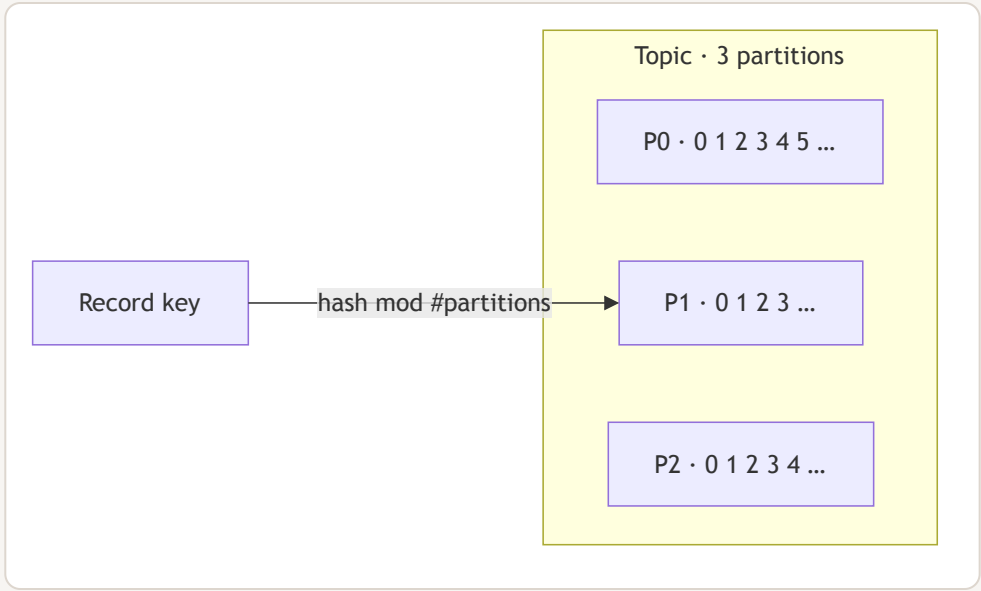
Partition log — offsets



Immutable sequence · sequential **offset** ids · older ← left · newer → right.

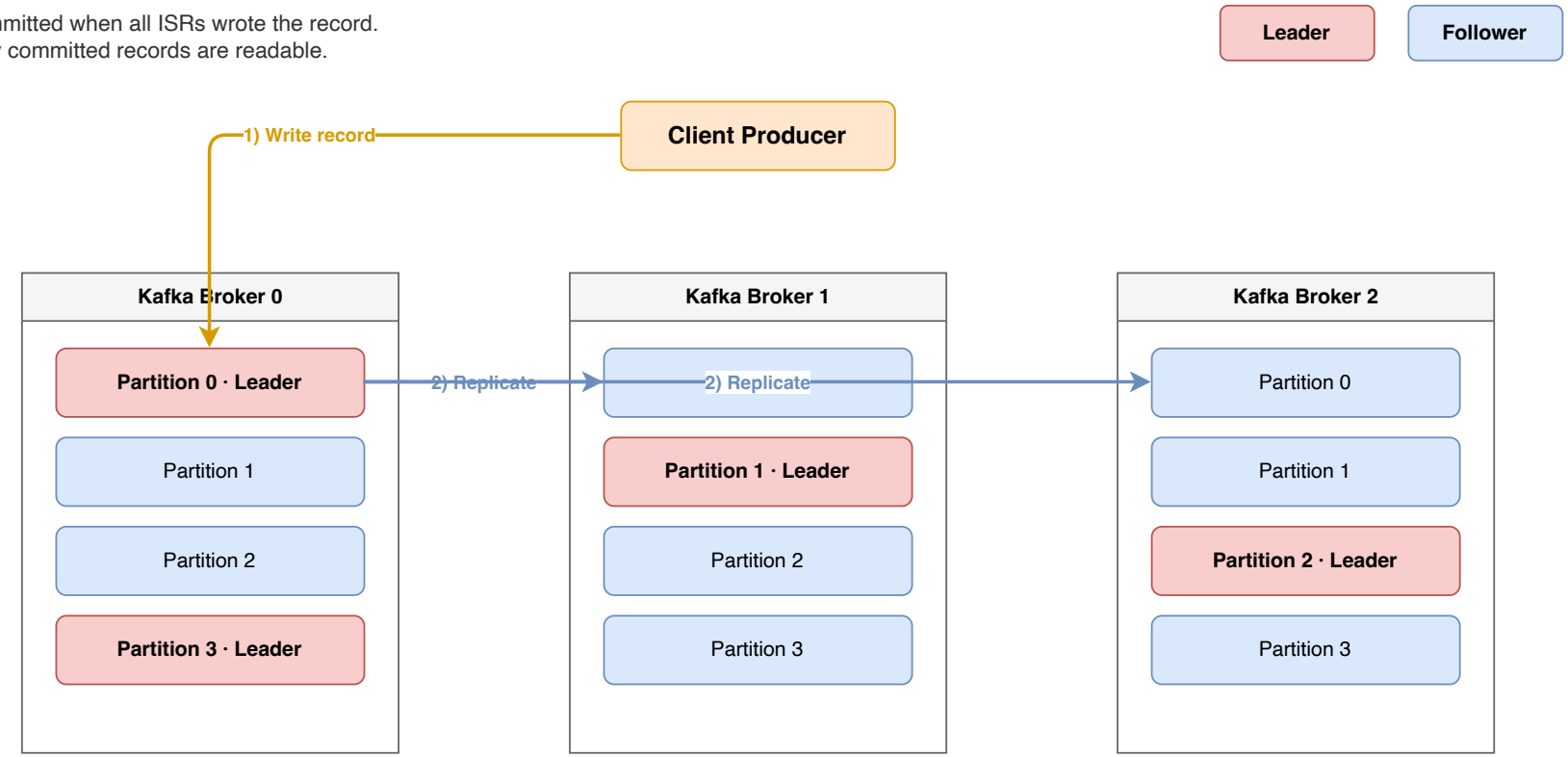
Partition layout (mental model)

- **Topic** encloses partitions
- Key → `hash mod #partitions` → **one** partition
- Same key → same partition → **per-key order**
- Retention: time / size / compaction



Replication — leader & ISR

Committed when all ISRs wrote the record.
Only committed records are readable.

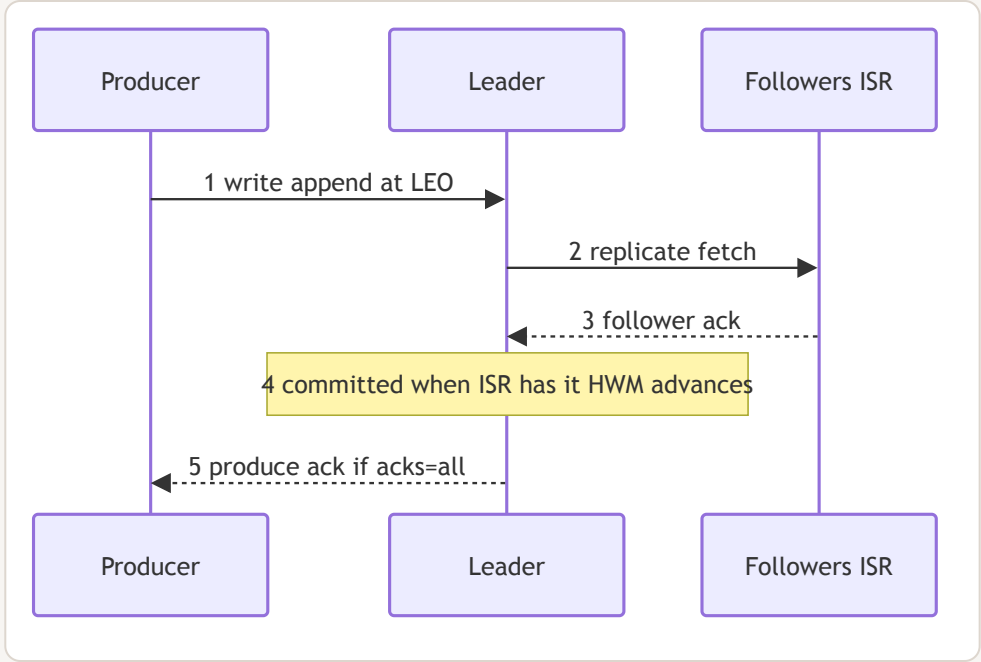


Replication write path

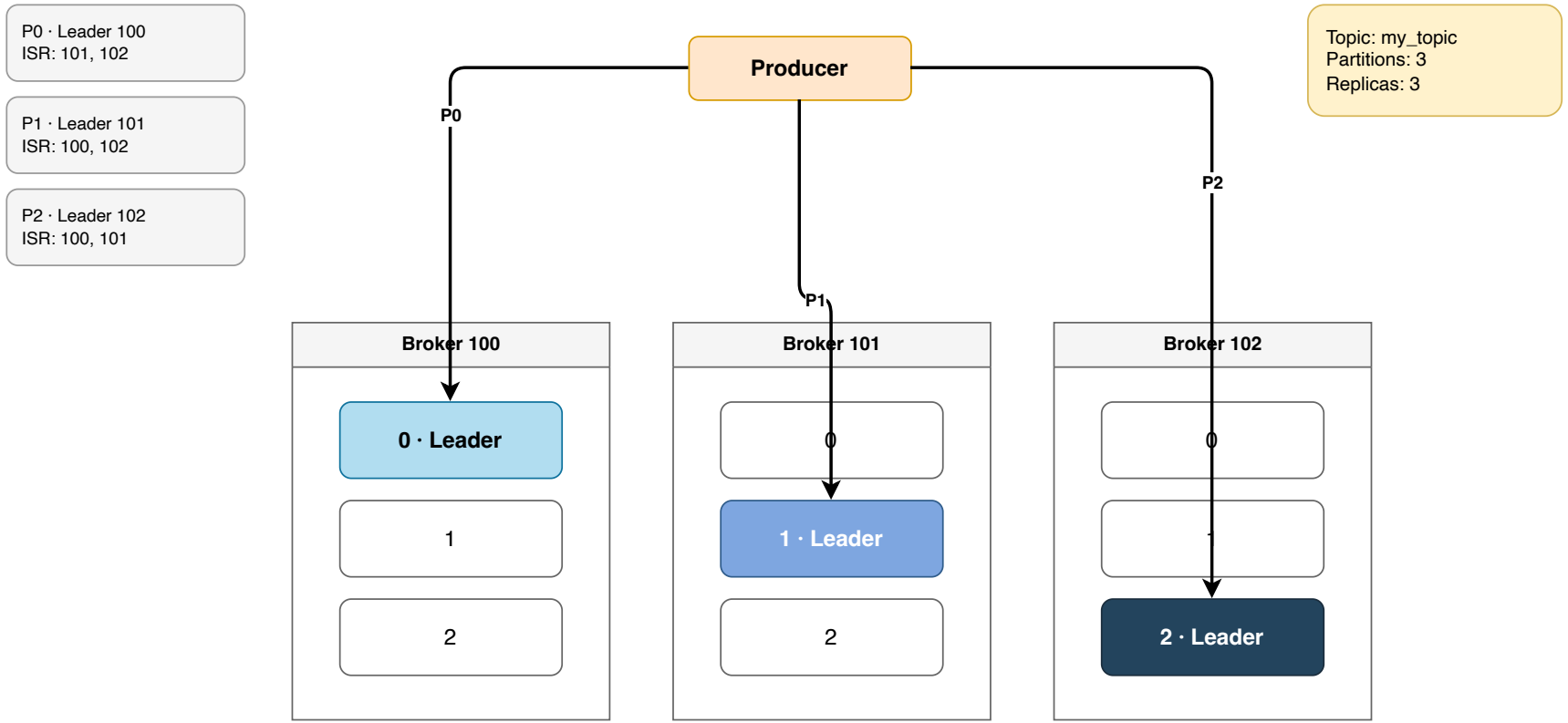
`acks=all` / `-1` **(durable)**

1. Write → **leader**
2. ISR **replicate**
3. Followers **ack** leader
4. **Committed** (HWM↑)
5. Produce **ack** → producer

`acks`	**When producer is acked**
`0`	Immediately
`1`	After **leader** has it
`all`	After **ISR** has it



Replication — leaders per partition



Guarantees (core)

- Messages from one producer to one partition arrive **in send order**
- Consumer sees log **in stored order**
- `min.insync.replicas` + producer `acks` control durability under failure

Retention

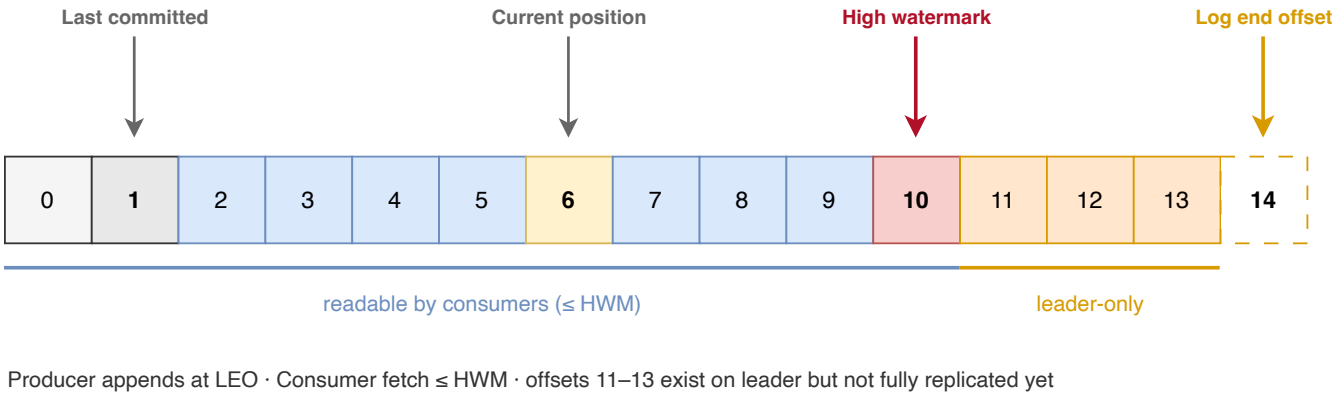
Policy	Behavior
Time	Keep for N hours/days
Size	Cap bytes per partition
Compaction	Keep latest value per key

Consumption speed ≠ retention size — old data stays until policy discards it.

High watermark (HWM)

LEO = next write · **HWM** = last offset fully in **ISR** · consumers fetch ≤ **HWM** (never leader-only).

Marker	Meaning
Committed / Current	Group-saved offset · where this consumer is reading
HWM / LEO	Last fully replicated (safe) · next offset producers append



Producer appends at LEO · Consumer fetch ≤ HWM · offsets 11–13 exist on leader but not fully replicated yet

Why Kafka is fast (intuition)

Lever	Effect
Sequential writes + page cache	Disk path stays hot; avoid random I/O
Batching + compression	Amortize network & syscall cost
Zero-copy (`sendfile`)	Fewer buffer copies broker → socket
Partition sharding	Many producers/consumers in parallel across brokers

Start with **≥3 brokers**; grow partitions/brokers for throughput — not one giant unpartitioned log.

6.3 Publisher (producer)

Producer responsibilities

- Choose **topic** (+ optional key)
- **Partitioning**: round-robin or key-based (“all events for `employeeId` together”)
- Trade **throughput vs durability** via `acks`
- Batch records for efficiency

Metadata: any broker can tell clients who leads which partition.

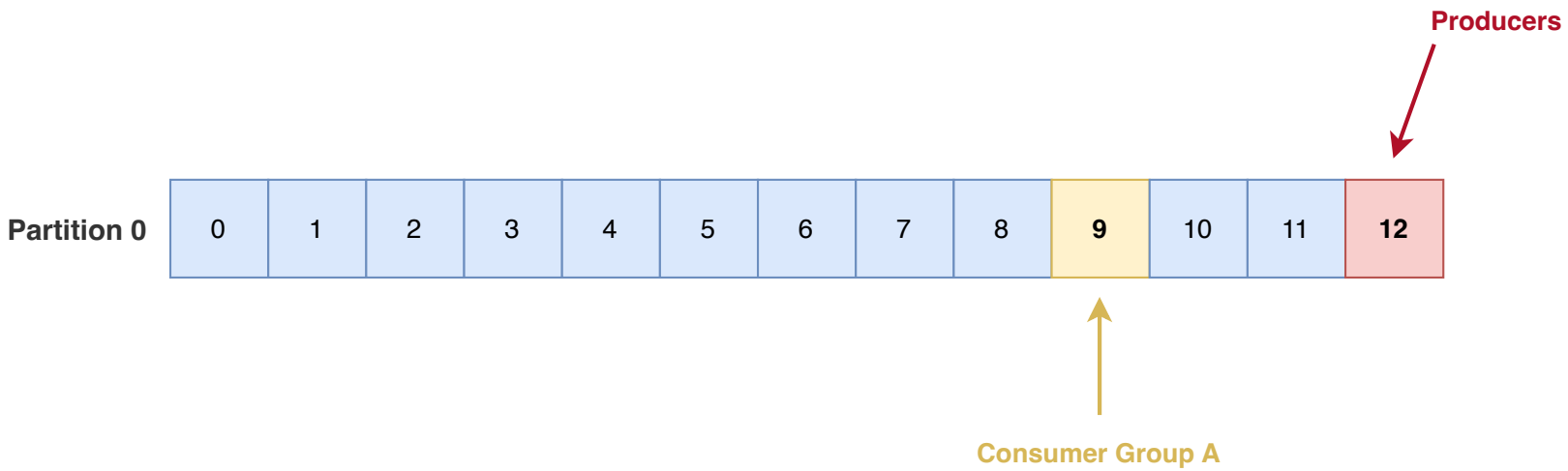
Durable writes — `acks`

Durability	`acks`	Behavior
Highest	`-1` / `all`	Wait for ISR
Medium	`1`	Leader only
Lowest	`0`	Fire-and-forget

Example hardening: `replication=3` , `min.insync.replicas=2` , `acks=-1` .

Producers & consumers on the log

Same partition log · producers **append at the tip** · each consumer group keeps **its own** read offset.



Producers append at offset 12 · Consumer Group A reads at offset 9 · each group keeps its own cursor

6.4 Consumer

Consumers own offsets

Messages **stay** until retention · resume from last **committed** offset.

1. Fetch & **commit** — consumer owns its progress on the partition
2. Leave / crash — records remain on the log
3. Rejoin → resume last commit → fetch the rest

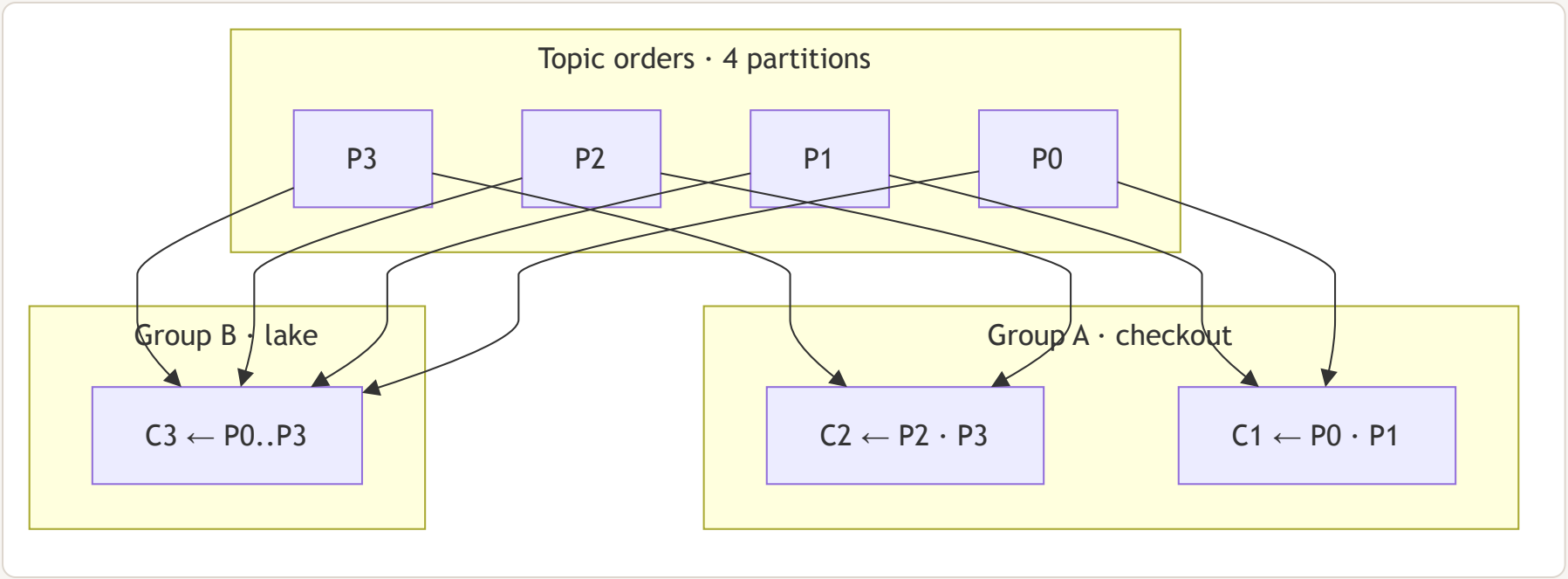
Crash **before** commit → replay → design **idempotent** handlers (at-least-once).

Consumer groups

Idea	Effect
One group	Queue-like load balance (each record → one member)
Many groups	Broadcast / fan-out (each group gets all records)
Group id	Independent subscribers (apps vs lake ingest)

Within a group: **one consumer per partition** (extras idle).

Groups — partition assignment map



Rebalance & failover

Membership change → **rebalance** (new assignment) · crash mid-work → peer resumes **last commit** (at-least-once).

Before — Group A (2 members)

C1 ← P0, P1

C2 ← P2, P3

After — C2 fails / leaves

C1 ← P0, P1, P2, P3

C2 ← offline

Rebalance pauses briefly, then reassigns every partition in the group.

Trigger	What happens
Join / leave / scale	Group rebalance → new partition assignment
Crash before commit	Peer takes partitions · resumes last committed offset · may **replay** → idempotent handlers

Partition ↔ consumer cardinality

- `#consumers > #partitions` → idle consumers (standby)
- `#partitions > #consumers` → some consumers read **multiple** partitions
- Scale consumers by **adding partitions** (plan ahead — hard to shrink)

Delivery semantics

Semantic	Meaning	Notes
At-most-once	May lose; never redeliver	Rarely desired
At-least-once	Never lose; may redeliver	**Default** practical path
Exactly-once	Once only	Idempotent produce + transactional / dedupe

Default path: design handlers **idempotent** under at-least-once.

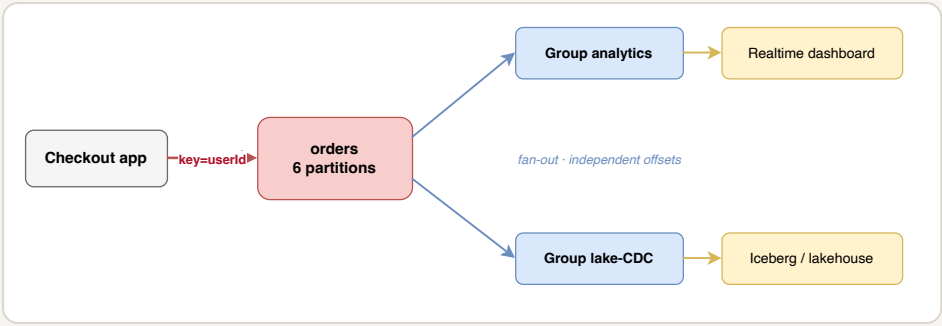
Exactly-once — what must line up

Side	Failure mode	Mitigation
Produce	Request sent; ack lost on network	Idempotent producer / transactional producer; check last committed offset
Consume	Processed but commit offset failed	Store **offset with output** (txn) or **dedupe** by record UUID
End-to-end	Both sides	Kafka transactions (read-process-write) or app-level idempotency

*“Exactly once” is a **protocol + app design**, not a checkbox on the broker alone.*

Walkthrough — topic `orders`

One durable event stream · many independent subscribers — glue the lecture ideas together.

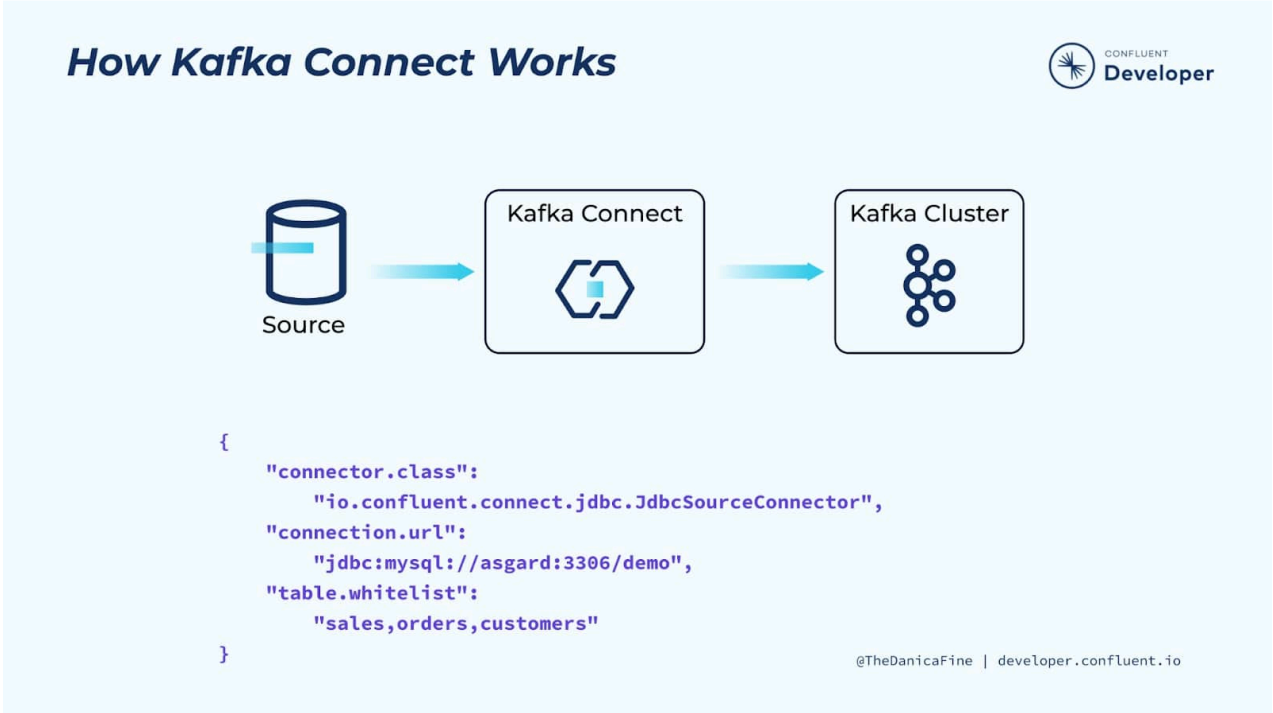


Choice	Why
Key = `userId`	Per-user **order** in one partition
`acks=all` + MIR=2	Durable checkout events
Two **groups**	Own offsets (fan-out)
Crash mid-batch	Replay → **idempotent**

6.5 Ecosystem — Connect · ksqlDB

Kafka Connect

External workers for **source → Kafka → sink**; scalable, fault-tolerant, declarative connectors.

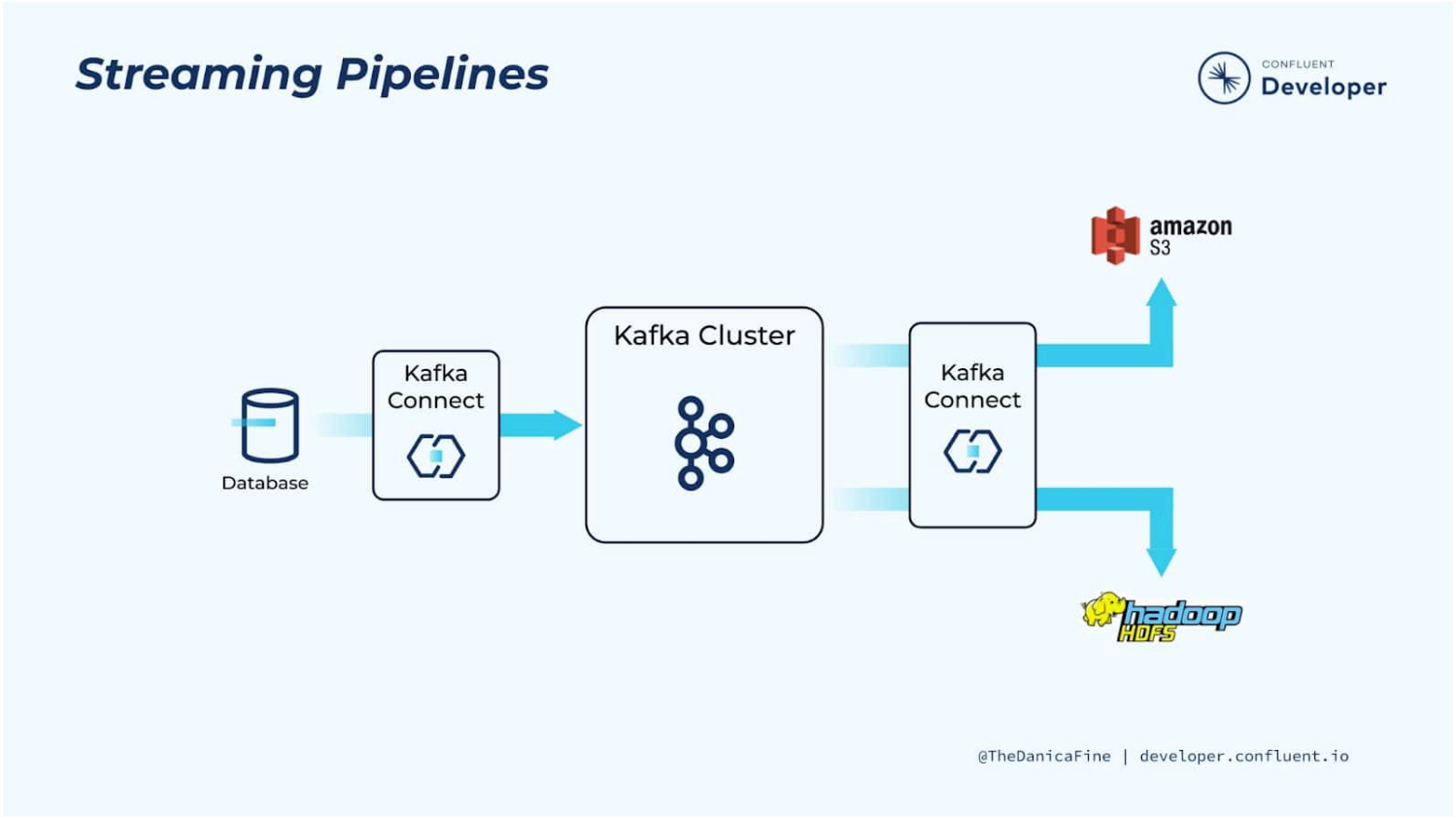


Connect ecosystem & pipelines

PPTX “ecosystem” slide was a **logo wall** — teach categories, not brands.

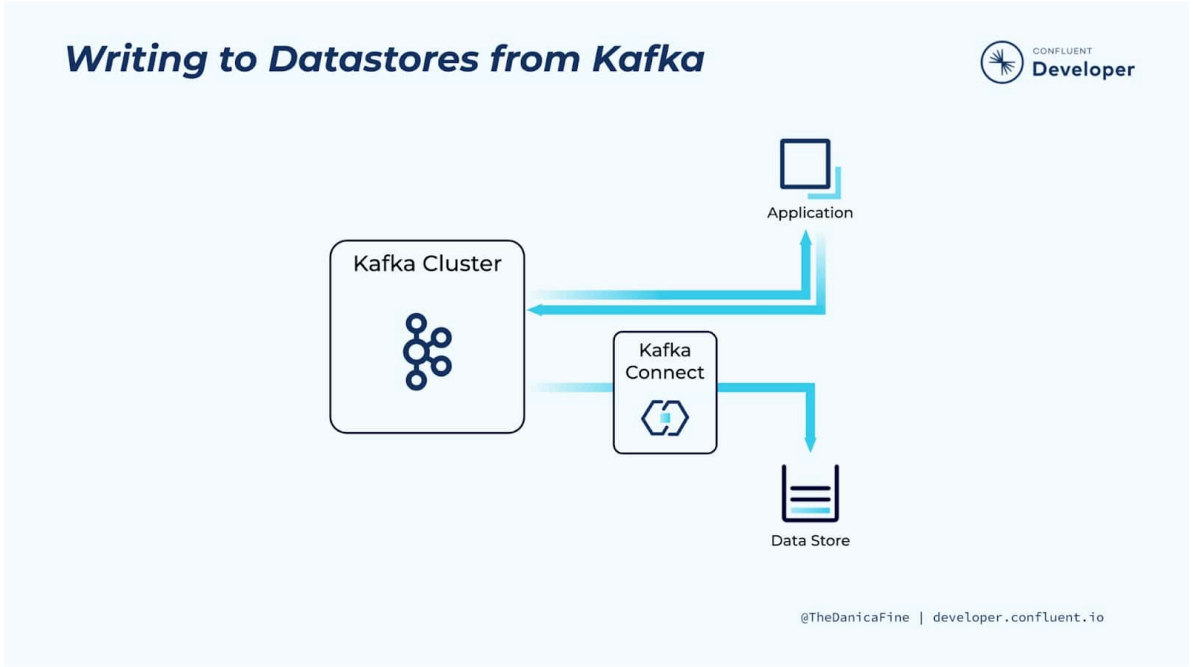
Side	Connector families (examples)
Sources	JDBC · **Debezium CDC** · files / HTTP · message queues
Kafka	Topics = durable log between producers & sinks
Sinks	JDBC · object store / HDFS / Iceberg · search · warehouses

Streaming pipeline shape

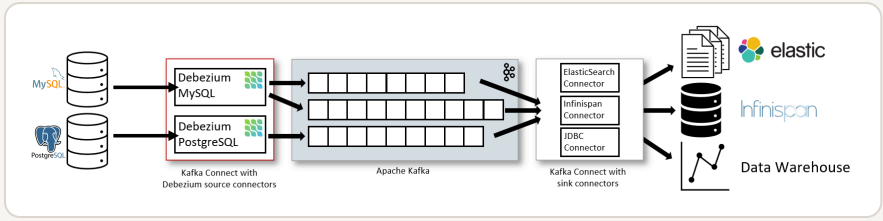


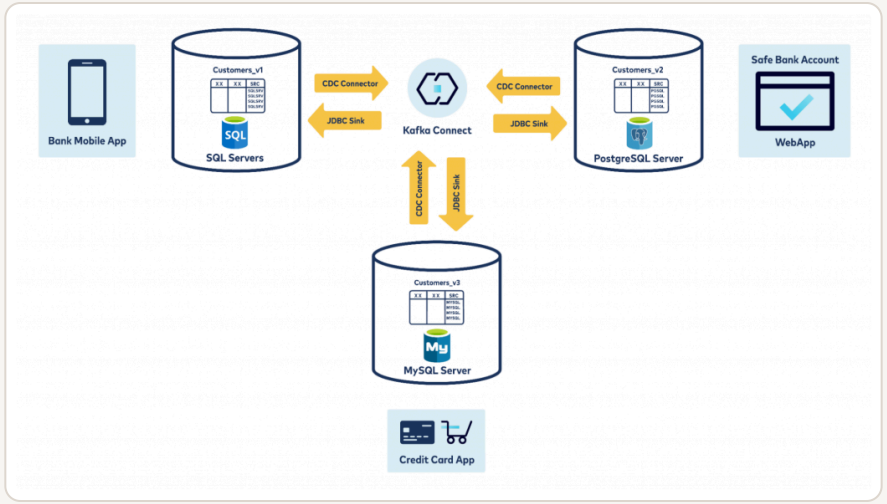
Writing to datastores from Kafka

Sink connectors push topics into search, warehouses, object stores — declarative, not custom consumers for every sink.



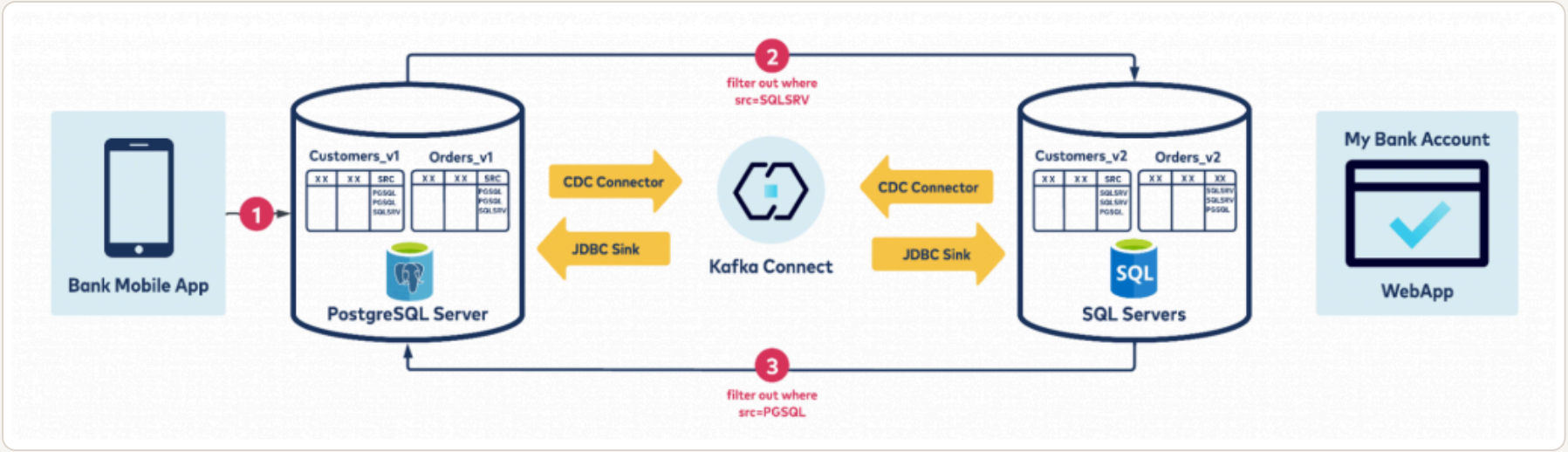
CDC & multi-DB sync





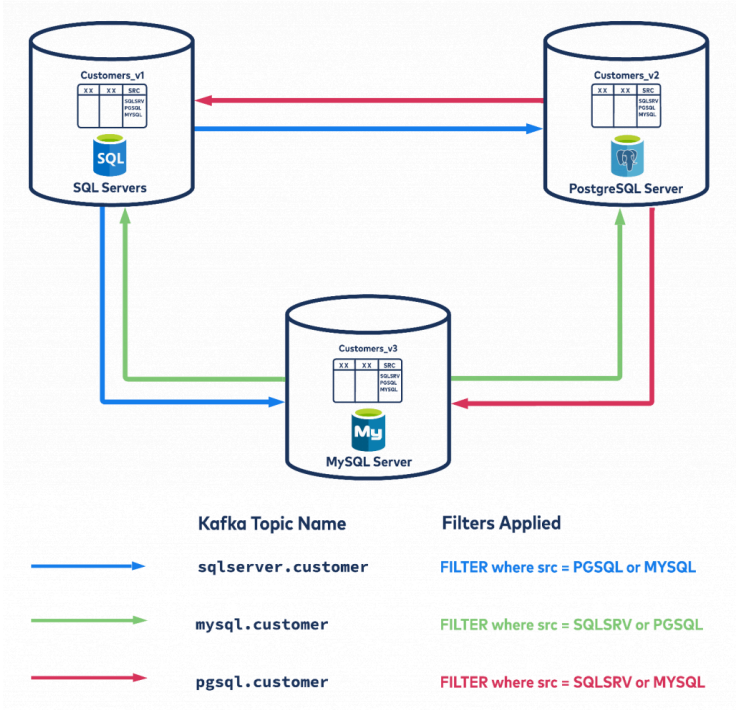
Sync two databases

Connect keeps two stores aligned via Kafka topics — source on one side, sink on the other.



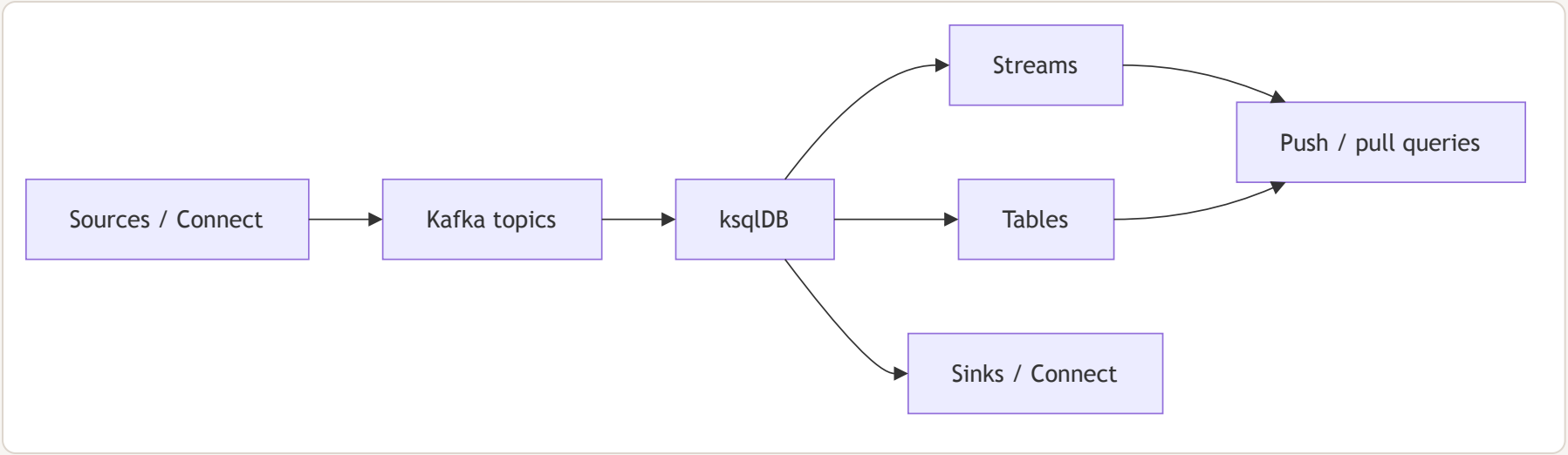
Filter at the source

Filter / route on the source connector or intermediate topics before sinks — less junk downstream.



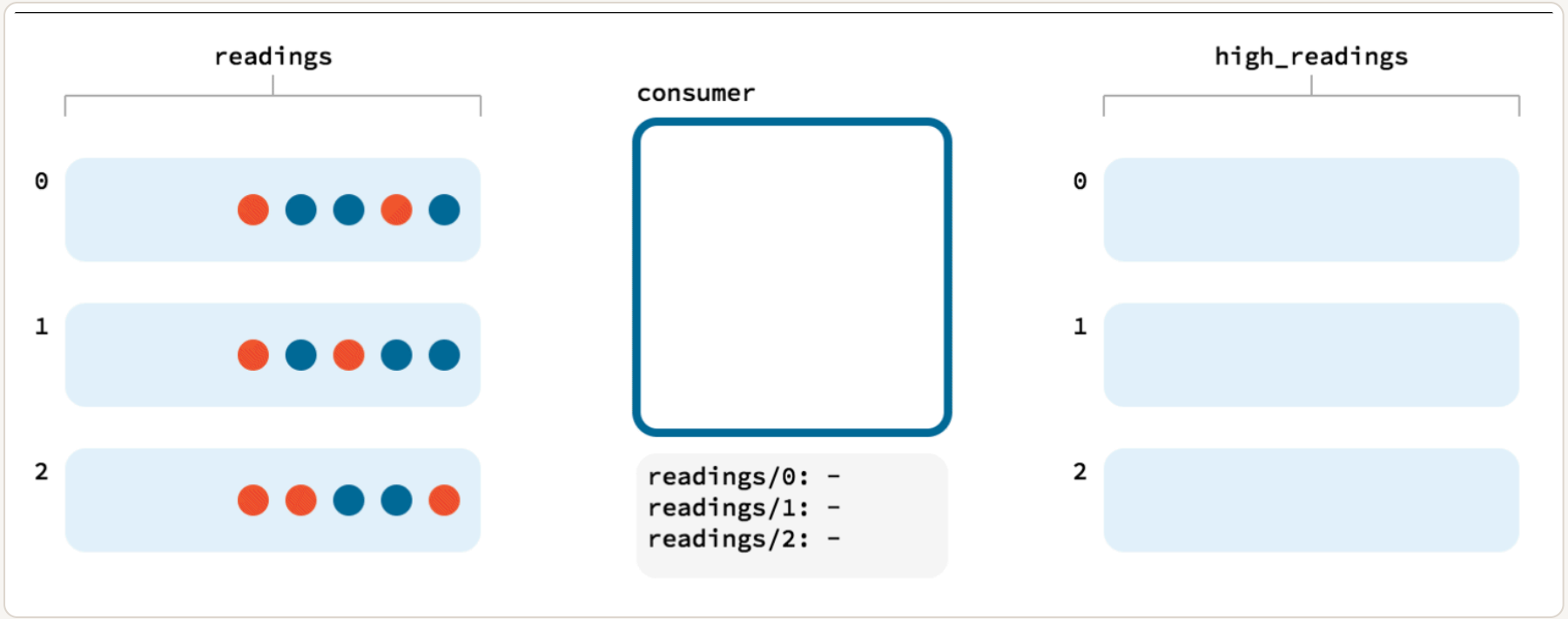
ksqlDB — SQL on streams

- Filter / join / aggregate with SQL (`EMIT CHANGES`)
- Push & pull queries; UDFs; connectors in one place
- Bridge toward **L11** stream processing



ksqlDB — filter stream

Push matching events to a new stream — SQL over partitions, not batch ETL.



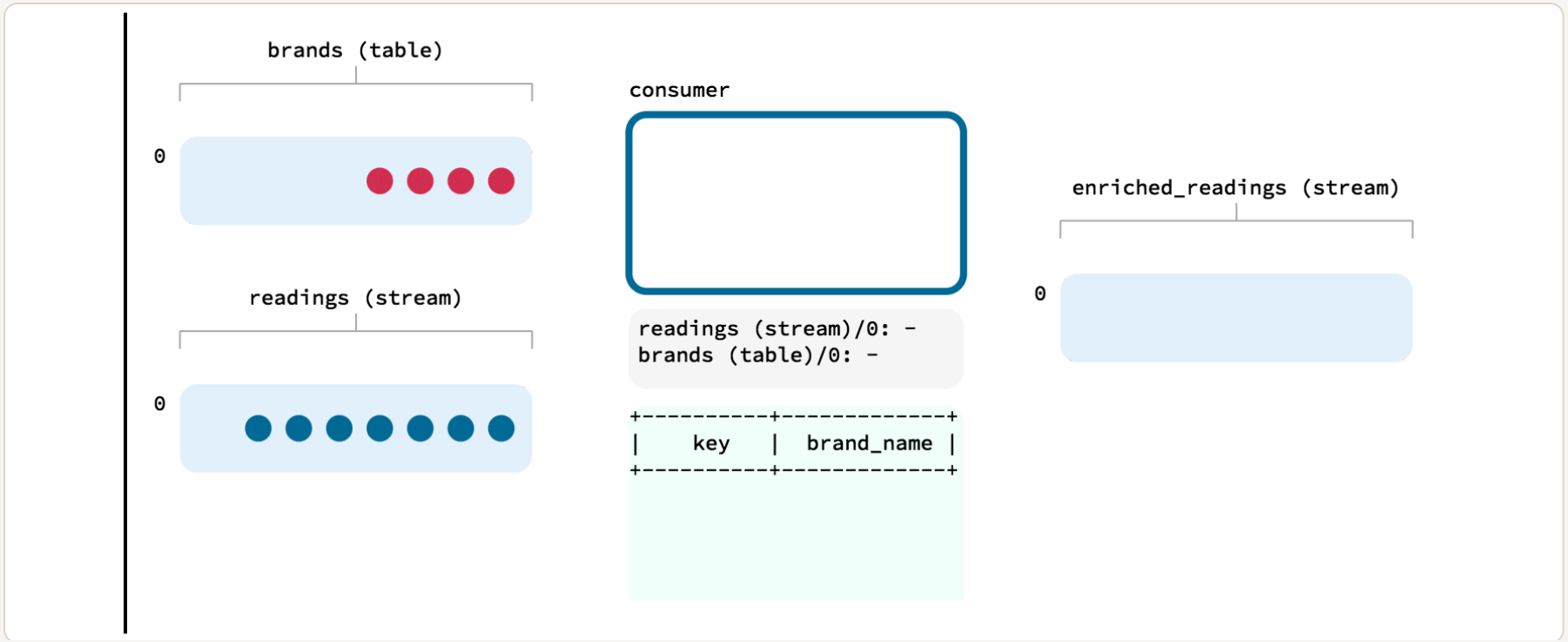
ksqlDB — filter (SQL)

```
CREATE STREAM high_readings AS
  SELECT sensor,
         reading,
         UCASE(location) AS location
  FROM readings
  WHERE reading > 41
  EMIT CHANGES;
```

Continuous query: matching records become a **new stream** (`high_readings`).

ksqlDB — join streams

Enrich a stream with another (e.g. readings ⋈ brands) — SQL join over topics.



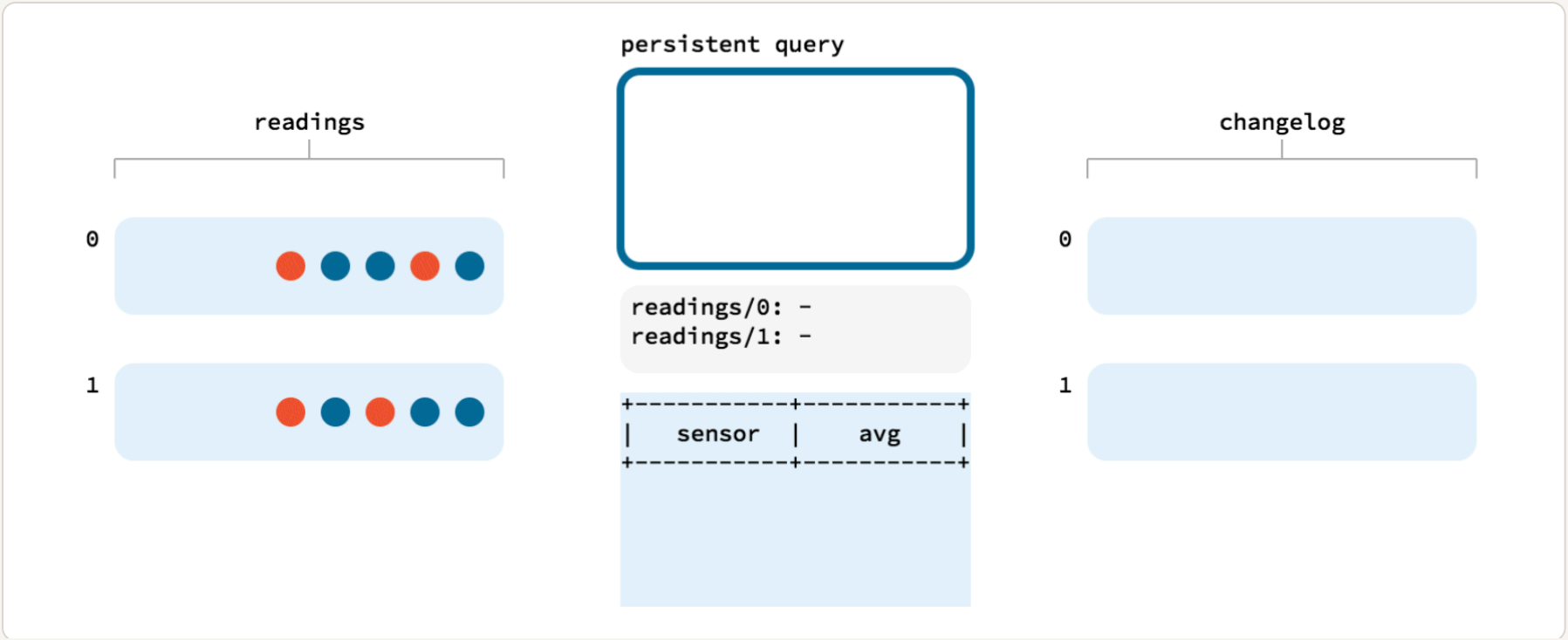
ksqlDB — join (SQL)

```
CREATE STREAM enriched_readings AS
  SELECT r.reading,
         r.area,
         b.brand_name
  FROM readings r
  INNER JOIN brands b
    ON b.sensor = r.sensor
  EMIT CHANGES;
```

Stream–table / stream–stream joins keep enrichment **in motion**.

ksqlDB — aggregate to table

`GROUP BY` turns a stream into a **table** for push/pull queries.



ksqlDB — aggregate (SQL)

```
CREATE TABLE avg_readings AS
  SELECT sensor,
         AVG(reading) AS avg_reading
  FROM readings
  GROUP BY sensor
  EMIT CHANGES;
```

`GROUP BY` + `EMIT CHANGES` → **changelog table** keyed by `sensor` (push/pull friendly).

Positioning (when not Kafka)

Fit	Not a great fit
Event messages / logs at scale	Huge file transfer as blobs
Replace many MQ use cases	Heavy in-broker transforms alone
Decouple microservices & lakes	Use current Kafka security (ACLs, mTLS, encryption) — do not assume an open cluster

Takeaways

1. Kafka = distributed **append-only event log**
2. **Partitions** → scale + per-key order; **ISR** → durability
3. Producers: keying + **acks**; consumers: **groups + offsets**
4. Default **at-least-once** → idempotent handlers; EOS needs both sides
5. Connect / CDC / ksqlDB → integration & stream SQL (toward L11 Spark SS)

Next

L09 — Big-data architectures (Lambda · Kappa)